b4M:
Holistic Benchmarking for MPC

Karl W. Koch[1,2],
Christian Rechberger[1] Dragos Rotaru[3],

[1] Graz University of Technology, Austria
christian.rechberger@tugraz.at
karl.koch@tugraz.at
[2] Secure Information Technology Center Austria (A-SIT), Graz, Austria
[3] Circle Internet Financial, New York, USA
dragos.rotaru@circle.com

Abstract. Secure Multi-Party Computation (MPC) is becoming more and more usable in practice. The practicality origins primarily from well-established general-purpose MPC frameworks, such as MP-SPDZ. However, to evaluate the practicality of an MPC program in the envisioned environments, still many benchmarks need to be done. We identified three challenges in the context of performance evaluations within the MPC domain: first, the cumbersome process to holistically benchmark MPC programs; second, the difficulty to find the best-possible MPC setting for a given task and envisioned environment; and third, to have consistent evaluations of the same task or problem area across projects and papers. In this work, we address the gap of tedious and complex benchmarking of MPC. Related works so far mostly provide a comparison for certain programs with different engines.

To the best of our knowledge, for the first time the whole benchmarking pipeline is automated; provided by our open-sourced framework `Holistic Benchmarking for MPC` (`b4M`). `b4M` is easy to configure using TOML files, outputs ready-to-use graphs, and provides even the MPC engine itself as own benchmark dimension. Furthermore it takes three relatively easy steps to add further engines: first, integrate engine-specific commands into `b4M`'s runner class; second, output performance metrics in `b4M`'s format; third, provide a Docker container for the engine's parties.

To showcase `b4M`, we provide an exemplary evaluation for the computation of the dot product and logistic regression using a real-world dataset. With this work, we move towards fully-automated evaluations of MPC programs, protocols, and engines, which smoothens the setup process and viewing various trade-offs. Hence, `b4M` advances MPC development by improving the benchmarking usability aspect of it.

Keywords: Multi-Party Computation · MP-SPDZ · webSPDZ · Benchmarking · b4M

The views expressed in this paper are solely those of the authors and should not be interpreted as reflecting the views of Circle Internet Financial or any other organization.

1 Introduction

The collected amount of data from, e.g., smartphones, smartwatches, IoT devices, or web applications is ever increasing. These data have the potential to learn a lot from them and thus to improve virtually all areas of our lives.One of the primary challenges is to protect the privacy of individuals. Secure Multi-Party Computation (MPC) is a technique to perform data analytics from various sources, while ensuring privacy of individuals.

MPC is increasingly usable in practice. The practicality origins primarily from mature MPC frameworks, such as "Multi-Protocol SPDZ - A Versatile Framework for MPC" (`MP-SPDZ`) [13,19]. However, we need to perform many benchmarks to evaluate the practicality of an MPC program in the envisioned environments.

Three Benchmarking Gaps. We identified three major challenges in the context of MPC performance evaluations. (1) Holistic MPC benchmarks: the cumbersome process to benchmark MPC programs for all possibly relevant settings. (2) Best-possible MPC setting: the difficulty to find the best MPC setting for a given task and envisioned environment. (3) Consistent MPC performance evaluations: to have consistent MPC performance evaluations of the same task/problem area across projects/papers.

As an example, without a loss of generality, we show a few MPC evaluations and their benchmark settings. In the area of privacy-preserving ML (PPML), one case of an MPC protocol dealing with rectified linear unit functions [2] and one dealing with general PPML by Dalskov et al. [8]. In the area of MPC-friendly PRFs, Hydra from 2023 by Grassi et al. [12] (Appendix J of the ePrint/full version shows the full benchmarks), Ciminion from 2022 by Dobraunig et al. [10], and an overview of a few PRFs by Grassi et al. [11]. Both Hydra and the overview of Grassi et al. focus on PRF evaluations with a secret-shared key, whereas Ciminion focuses on a theoretical comparison using number of multiplications. Further, Hydra gives a graph for comparing the number of multiplications for the branch size t with competitive PRFs.

Table 1 gives an overview of these example MPC evaluations and their benchmark settings.

Related Work. Lorünser and Wohner [18] compared the general-purpose MPC engines, `Multi-Protocol SPDZ` (`MP-SPDZ`) [13] and "Multi-Party Computation in Python" (`MPyC`) [22], on their usability and performance of symmetric-cipher evaluations. They concluded that practical performance of an MPC program is hard to estimate without implementing it. For instance, with respect to the MPC environment, `MP-SPDZ` performed better in fast networks, while `MPyC` outperformed `MP-SPDZ` for a small number of parties when the network latency increased.

Keller introduced `MP-SPDZ` [13], which provides several features: honest vs. dishonest majority, active vs. passive security, dynamic number of parties, and arithmetic as well as binary circuits for arbitrary MPC programs.

	MPC-based PPML		MPC-friendly PRFs		
Benchmark Parameter	**ReLU** Aly et al. [2]	**SecInf** Dalskov et al. [8]	**Hydra** Grassi et al. [12]	**Ciminion** Dobraunig et al. [10]	**CCS-16** Grassi et al. [11]
Evaluation measurements	Practical	Practical	Practical Theoretical	Theoretical	Pratical Theoretical
#Parties	2-3	3-5	2	-	2
MPC engine	`SCALE`	`MP-SPDZ`	`MP-SPDZ`	-	`SPDZ-2`
Online vs. Offline Phase	Online	Offline Online	Offline Online	Online	Offline Online
Protocol	HighGear, Shamir	LowGear, Replicated-Prime, etc.	MASCOT	-	MASCOT
Threat Model Corruption	Active	Active Passive	Active	-	Active
Threat Model Majority	Honest Dishonest	Honest Dishonest	Dishonest	-	Dishonest
Network LAN	$\perp$ $\perp$	*"sub"*ms $10Gbit/s$	«$1ms$ *rt* $1Gbit/s$	-	$\perp$ $1Gbit/s$
Network WAN	$10/20ms$ *ow* $\perp$	*"AWS"* *"AWS"*	-	-	$100ms$ $\perp$ $50Mbit/s$
Evaluation metrics	Runtime	Runtime Net. data	Runtime Net. data #Online rounds #Multiplications	#Online rounds #Multiplications	Runtime #Online rounds #Multiplications
Result Format	Tables	Tables Graphs	Tables 1 Graph	Tables	Tables
∅ #Runs	$\perp$	$\perp$	200	-	5
Computational threads	$\perp$	32 for `MP-SPDZ`	$\perp$	-	Single-threaded
Branch Size t *(for PRFs)*	-	-	8,16,32, 64,128	Generic	$\perp$

Table 1: Overview of a example performance evaluations of MPC cases and their benchmark settings. The "-" symbol denotes that the respective benchmark parameter is not applicable to the respective use case/evaluation; e.g., #Parties in a theoretical-only evaluation, or if an evaluation has not used a WAN network speed. The "$\perp$ " symbol denotes an unknown value for the given evaluation. For the networks, the first and second row denote latency and bandwidth respectively. W.r.t. latency, "ow"- and "rt"-latency denote one-way- and round-trip latency respectively. In the WAN-network case of **SecInf**, *"AWS"* denotes the setting where AWS machines on different continents are selected; with no concrete network numbers.

While `MP-SPDZ` does not provide a benchmarking tool for various settings of the same program out of the box, it shows the complexity of the many run options for MPC. In the introductory paper, Keller compares `MP-SPDZ` with 8 other MPC engines for the computation of the dot product of two vectors.

Barak et al. [4] introduced an end-to-end automated system for deploying large-scale MPC programs between end users, dubbed MPSaaS (MPC System as a Service). Primarily, they provide an MPSaaS software where users can actively participate in MPC computations and show a new MPC protocol. Barak et al. evaluated their protocol with 10–150 parties, in increments of 10, for different program parameters (circuit sizes and depths for multiplications). Since the corresponding GitHub project [7] shows only the project's web app (frontend), it is unclear which MPC engine they used for their evaluation. Thus, their experiments cannot be reproduced in a straightforward way.

Take Aways. Many works consider MPC benchmarking in one way or the other. Due to page limitation, we showed some related work close to our gap analysis. The related works show that holistic MPC performance evaluations can be complex (various programs, engines, security models, number of parties, network types, etc.). Moreover, the performance evaluations are non-trivial to forecast and sometimes hard to reproduce. We suspect that a performance forecast becomes even harder when we consider all possibly relevant run settings. Further, these points highlight the importance of reproducibility for MPC benchmarks. Thus, we underline that holistic MPC benchmarks are cumbersome to achieve in practice and are worth to investigate.

Our Contributions. We address the three gaps by providing the open-source framework `Holistic Benchmarking for MPC` (`b4M`)[4]. (1) Holistic MPC benchmarks (all possibly relevant settings, including even the engine as parameter), (2) best-possible MPC setting (by analyzing the holistic benchmarks), and (3) consistent MPC performance evaluations (across different papers). Thus, `b4M` enables a smooth way to perform holistic MPC benchmarks of envisioned scenarios with requirements in mind, and provides ready-to-use graphs: (i) select the scenario/settings ⇒ (ii) run the program ⇒ (iii) get the graphs. As such, `b4M` enhances MPC's development and practical research aspects.

Additionally, we show `b4M`'s BenchFlow for two exemplary scenarios. "Party Engines", which evaluates the dot product for various number of parties using two MPC engines. "Protocol Epochs", which performs logistic regression on exemplary breast-cancer data using six protocols which provide different levels of security.

Outline. Section 3 describes the `b4M` framework and how the whole BenchFlow works. Section 4 showcases the BenchFlow with the exemplary programs dot product of two vectors and logistic regression on breast-cancer data. Further, Section 2 gives supporting preliminaries and Section 5 concludes this work.

[4] github.com/kaydoubleu/b4M

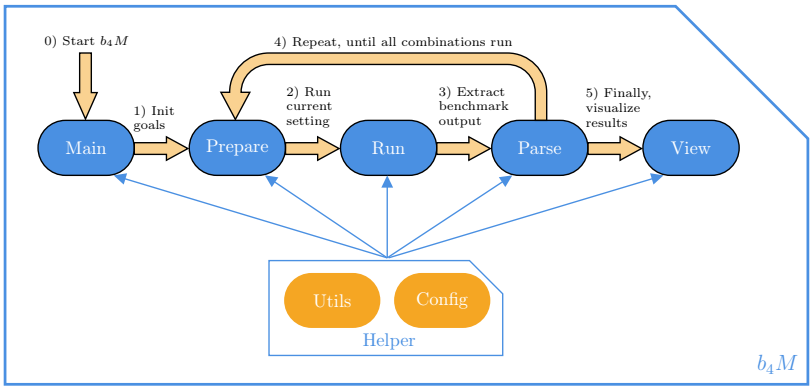

Fig. 1: `b4M`'s higher-level building blocks within the framework core (Python) and their position within the pipeline. Main modules are in blue and on the top. Helper modules are in orange and on the bottom.

2 Preliminaries for b4M

Following the notations of Buchsteiner et al. [5], in this section, we briefly describe MPC with the related security models and protocols. Moreover, we define "MPC engine" and "holistic MPC benchmarking". For instance, Smart [25] and Lindell [17] give further general information on MPC.

MPC enables various parties to jointly compute a function over secret inputs in a secure way. No party learns other inputs or intermediate results. Function outputs are delivered to all or a subset of the parties. MPC computations use an MPC protocol under a specific security model, running within an "MPC engine".

MPC Protocols & Security Models. To run an MPC program, one selects an underlying protocol depending on the required level of security and performance. In terms of security, one usually decides how many parties can be corrupt (majority) and the corruption type. The corruption majority classifies primarily into two categories: (M-i) honest majority, where at most half of the parties are corrupt and (M-ii) dishonest majority, which allows up to all but one corrupted parties. The corruption type classifies primarily into two categories as well: (C-i) passive corruption (semi-honest security), where corrupted parties learn from the protocol transcript while following the protocol's instructions and (C-ii) active corruption (malicious security), where corrupted parties additionally might deviate from the protocol (e.g., sending malformed data).

MPC Engines. We denote "MPC engine" as a framework that enables parties to actively participate in MPC computations. Further, we denote it as general-purpose MPC engine if it supports arbitrary computations and a dynamic amount of participating parties. In the popular example `MP-SPDZ` [14], the MPC program is described in a high-level Python-like language (frontend).

Then, the high-level code gets compiled down to a series of basic instructions, such as additions or multiplications [16]. Finally, a (backend) virtual machine instruments these instructions and jointly performs the computations with the other parties' engines.

Holistic MPC Benchmarking. We define holistic MPC benchmarking as taking all relevant MPC run settings into account. On the one hand, environment settings: number of participating parties, network (latency, bandwidth), and computational resources (RAM, CPU). On the other hand, program settings: engine, protocol (majority, corruption), input parameters, and number of run iterations. Moreover, inspecting relevant metrics: runtime, communication (number of rounds, network data), and memory consumption (peak RAM). Ideally, depending on the use case's needs, these metrics can be split into "micro benchmarks", which allows for fine-grained analyses of different program parts.

3 b4M Framework: Holistic Benchmarking for MPC

In this section, we describe the holistic MPC benchmarking framework `b4M`. Initially, we give an overview of the framework's stack, supported dimensions, and performance metrics. Then, we describe the two primary phases of the BenchFlow: "Select & Run" and "Analyze & Conclude".

3.1 Framework Stack

The core and engine integration. `b4M` is based on the programming language Python. Figure 1 shows the main building blocks on a higher level and their position within the flow's pipeline. Each MPC engine is set up in a Docker container.

To run the individual engines, `b4M` uses the Python module `docker` to create and manage a Docker container for the engine's player. Specifically, at the core of the engines' executions is a runner class which includes dedicated methods for the individual engines. These methods perform the engine-specific run preparations. From the outside of this runner class, there's an initialization and a generic runner method. After an engine's benchmark run, the Docker's log output is redirected to a generic log destination. Everything before these runner-class methods and after the generic log output is engine-agnostic. With this single point of contact in the runner class, we aim to ease the integration of further engines as much as possible.

Setup. A convenient way to install all Python requirements is to use `pipenv`. Then, to perform the benchmarks: we first need to modify the `TOML` configuration file and then run `b4M`'s main entrypoint.

The TOML configuration file includes the settings of the active dimensions to benchmark and view, engine-dependent information such as paths and run commands, general run settings (e.g., the number of runs per benchmark or timeout), and the setting of what to display in the runtime and network-data graphs, e.g.,

number of players on the X-axis with the runtime unit minutes on the Y-axis. This way we aim to have the setup of `b4M` as easy as possible. Figure 2 shows the three phases, with accompanying terminal commands.

For the setup of the network speed, `b4M` uses a Docker network with a dedicated IP range for each network setting. These Docker networks have to be created manually in advance, as of now. Then for each network setting, the respective latency and bandwidth is set in a network-setup script. `b4M` provides such a network-setup script, which emulates the chosen network speed for a dedicated IP range; e.g., 100ms round-trip latency with 100MBit/s bandwidth.

3.2 Supported Dimensions

`b4M` provides the following steps in the process of MPC benchmarking:

1. **Benchmark Execution**
   (a) Select various benchmark settings
   (b) Benchmark all combinations of selected benchmark settings
2. **Benchmark Evaluation**
   (a) Select relevant metrics
   (b) Investigate benchmark results via graphs or the raw outcome *(JSON)*

`b4M` allows to benchmark a given MPC program with various settings, which we call benchmark dimensions. We split the dimensions into two main categories:

1. **Environment-specific**
   - **MPC engine:** `MP-SPDZ`, `webSPDZ`, `MPyC-Web`, ...
   - **Number of parties:** 3, 4, 5,... 10,...
   - **Network speed:** fast/unrestricted *(LAN)* and slow/restricted *(WAN; e.g., 100ms round-trip latency & 100Mbit/s bandwidth)*
   - **Threat model:** corruption model *(semi-honest/malicious)* and trust majority *(honest/dishonest)*
   - **Underlying MPC protocol:** Shamir [23], Replicated [20], ...
2. **Program-specific**
   - **Program parameters:** e.g., multiplicative depth, vector length, or the branch size of a pseudo-random function (PRF)
   - **Datasets:** e.g., inputted dataset/tables with 32/64/128/etc. entries

After the setup and runs of the various dimensions, `b4M` gives a detailed overview of the results. Result measurements are shown for one party of the whole run; hence, it includes both the offline & online phase, if not chosen otherwise. If more fine-grained analyses are necessary, we can measure the time of different sections in the MPC program.

The supported values of `b4M`'s benchmark dimensions depend mostly on the underlying MPC engines. However, the network speed dimension is engine-independent. For the network speed, `b4M` currently supports a LAN and WAN environment, where WAN has 100ms round-trip latency and 100Mbit/s bandwidth. We can adapt the latency and bandwidth by using the network-setup script within the engines' Docker containers.

```
(0) __Install Python requirements__
  (a) $ pipenv install
  (b) $ pipenv shell
(1) __Select Setting__
  (a) # What to benchmark
      # (TOML config; e.g., RUN=true, #Players=[3,4,5])
  (b) # What to view
      # (TOML config; e.g., VIEW=true, X-axis=#Players)
(2) __Perform benchmarks & Generate results__
  (a) $ python3 b4M_main.py
      # benchmarking & viewing can be done separately
      # (e.g., RUN=true & VIEW=false in TOML config)
  (b) # Investigate graphs
```

Fig. 2: Three phases to apply the full pipeline of `b4M` with examples of how to perform the pipeline phases. Lines with two underlines *(__)* denote a a phase. Lines with a *"$"* symbol denote a terminal command within the main folder of `b4M`. Lines with a *"#"* symbol denote a comment.

Moreover, one of the core features of `b4M` is the independence to specific MPC engines. Any MPC protocol can be used to make benchmarks with `b4M`. Thus, `b4M` provides even the MPC engine as own benchmark dimension.

To achieve this the MPC engines need to output the desired performance metrics in `b4M`'s format (see below) and add MPC-engine-specific run commands in `b4M` (see above). In addition, it is also required to provide the MPC program in the respective MPC-engine's front-end language. However, these changes and adaptions are relatively minor and `b4M` is designed in a way that MPC engines can be added easily and the whole benchmark flow is as MPC-engine agnostic as possible.

Please note that `b4M` can, of course, only show what the underlying MPC engine(s) support. For instance, if `MP-SPDZ` does not support the output of number of multiplications, `b4M` cannot show this metric. Though, `b4M` virtually supports any underlying MPC engine. An MPC engine *only* needs to output the desired performance metrics in the required format; and, ideally, provides a respective Docker container. Hence, `b4M` is MPC-engine-agnostic.

Performance metrics. To capture all mentioned performance metrics, the MPC engine needs to support the respective output. For instance, for the time metric in `MP-SPDZ`, we need to add a start and stop timer command in the MPC program. Figure 3 shows an example timer for the dot product in `MP-SPDZ`.

Currently, `b4M` supports performance metrics for runtime and communication. For the runtime, even various timings for different program sections can be displayed. For communication, the amount of sent and received bytes can be displayed depending on the engine support. The sent and received bytes are the maximum value per player of all players, averaged over multiple runs.

```
1    start_timer(1)
2    result = sint.dot_product(a, b)
3    stop_timer(1)
```

Fig. 3: `MP-SPDZ`'s syntax in the MPC program to activate the benchmarking output for the time metric. In this example, the timer has the index 1. We can add various timers for different program sections (distinct timer index). The program part itself (Line 2), shows how to compute the dot product in `MP-SPDZ`.

Current MPC-engines support. There exist many useful, and relatively mature, MPC engines. For instance, the Awesome MPC list [21] gives an overview of various (active) MPC engines. Currently, we focus on general-purpose MPC engines. Concretely, `b4M` supports the "Native Variant of Multi-Protocol-SPDZ" (`natSPDZ`), the "Web Variant of Multi-Protocol-SPDZ" (`webSPDZ`), the "Web Variant of Multiparty Computation in Python" (`MPyC-Web`), and the "JavaScript library for building web apps that employ MPC" (`JIFF`). However, we can easily add other MPC engines due to `b4M`'s openness and flexibility.

3.3 Phase 1: Select the Setup and Run the Combination of Settings

In the first benchmarking phase, we select the benchmarking setup: the benchmarking strategy and other relevant benchmarking dimensions. Afterwards, `b4M` runs all combinations of the selected dimensions. Finally in this phase, `b4M` extracts all relevant information from the run logs.

Select. The benchmarking setup, strategy and dimensions, is selected by editing the respective `TOML` configuration file. Strategy-wise `b4M` supports, as of now, "exhaustive benchmarking". For this benchmarking strategy all combinations of selected dimensions are run.

Run. When we have selected the benchmarking setup, `b4M` provides one preferred way to run the benchmarks. The benchmarks run on the respective local machine, via the MPC-engines' Docker containers. Then for each engine's player, a respective Docker container is created. After each run the logs are created.

Capture statistics from the MPC engines. The last step of the first benchmarking phase is to capture the relevant run output. We envision a connection to engines which enforces as few changes as possible from the engines, while still having a smooth way to incorporate several engines. Thus, MPC engines need to output the desired information, such as performance metrics, in `b4M`'s parsing format. For the sake of parsing simplicity, we require `b4M`-related output as JSON within a HTML element. The HTML element lets us swiftly filter for relevant entries. And then, we load the HTML-element's content as JSON, and further process the respective metric output. Figure 4 shows an example output for the time metric.

The engine can adapt non-`b4M`-specific log output as desired without breaking the dependency to `b4M`. Hence, we aim for sustainability in the long run.

```
<b3m4>
{"timer":1,
  "time":{"sec":179.018120,"ms":179018.1199}
}
</b3m4>
```

Fig. 4: Example benchmark output of the runtime; here for timer index 1. The runtime is provided in the JSON object "timer", in the units seconds and milliseconds. As in all benchmark-relevant outputs, the JSON-formatted measurements are wrapped within b4M's magic HTML element <b3m4>.

3.4 Phase 2: Analyze the Output and Conclude

Once all combinations of settings have been run, we dive into the second benchmarking phase: analyze the output and draw conclusions (for, e.g., upcoming scenarios). The vision of b4M is to enable lessons learnt for potential next steps. The captured JSON output serves as starting point. As of now, we analyze the JSON output in two ways: either to investigate the b4M-created graphs or directly the JSON output.

b4M-created graphs. As for the first benchmarking phase, we select relevant dimensions and their values. As a prerequisite here, run logs for the selected benchmark setting need to exist. Then, we select which data we want to view in the respective graphs *(X-axis, Y-axis, content)*. We can create various graphs with different data-analysis aspects, based on the specific use case's needs. b4M creates graphs based on Python's `matplotlib` and `seaborn` library.

Draw conclusions. Which conclusions one draws depends on the use case. b4M provides a basis where we can look at all relevant dimensions and performance aspects. One example would be to check for the fastest protocols within a range of participating parties.

4 Exemplary Evaluation using b4M

In this section, we showcase b4M's BenchFlow for two exemplary scenarios. Initially, we describe our benchmarking approach. Then, we show our results.

In Scenario One, "Party Engines", we evaluate the dot product of two vectors for various number of parties using two MPC engines. In Scenario Two, "Protocol Epochs", we evaluate logistic regression on exemplary breast-cancer data using six protocols which provide different levels of security. Four protocols in honest majority: semi-honest and malicious Shamir [6,24,1], semi-honest Replicated for 3 parties by Araki et al. [3] and malicious Replicated for 4 parties by Dalskov et al. [9]. Two protocols in dishonest majority: active MASCOT [15] and passive semi2, which is essentially the "passive variant of MASCOT".

As general-purpose MPC engines, we use `natSPDZ` and `webSPDZ`. Table 2 shows the various dimensions of the two scenarios.

4.1 Benchmarking Environment

Each party runs in its own Docker container with a limit of 4GB RAM and 4 vCPU threads, on an *x86_ 64 AMD EPYC 7502 32-Core Processor* using Linux. `webSPDZ`'s containers simulate the web browser using Selenium in Python. We measure the runtime by averaging over three runs per benchmark combination.

4.2 Results

Figure 5 shows the dot product runtime for Scenario One, "Party Engines".

For Scenario Two, "Protocol Epochs", Figure 7 shows the logistic regression runtime using Replicated and semi-honest Shamir for 3 parties. Additionally, Figure 8 showcases only the evaluation runtime, after the training phase. Figure 6 shows the runtime using Replicated and malicious Shamir for 4 parties.

We ran the dishonest-majority protocols MASCOT and semi2 only for one training epoch of the logistic regression task since the runtime was relatively slow. MASCOT took overall 1,770s (∼30min) (∼25min training; ∼5min evaluation). semi2 took overall 350s (∼6min) (∼5.5min training; ∼0.5min evaluation).

Table 2: Selected dimensions and their values for our exemplary evaluation showcase.The scenario-specific focus points are highlighted in blue and separated with a slash *("/")* respectively.

Environment Dimension	Scenario	
	(1) "Party Engines"	**(2) "Protocols & Epochs"**
Engine	`webSPDZ`/`natSPDZ`	`natSPDZ`
Protocol	Shamir	Sh./Rep3./Rep4./semi2/MA.
#Players	3/4/.../9	3/4
Network Speed	LAN	LAN
Program Dimension		
Vector Length	100,000	-
Training Epochs	-	1/2/3/4/5

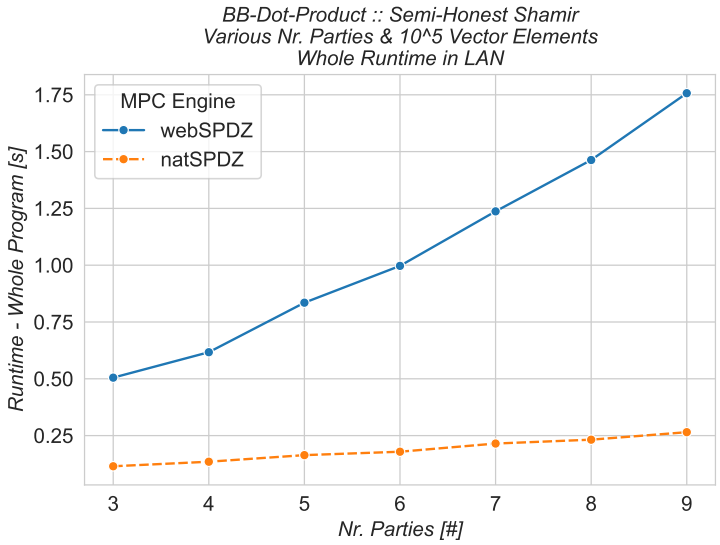

Fig. 5: Overall runtime in seconds for the computation of the dot product in Scenario One, "Party Engines".

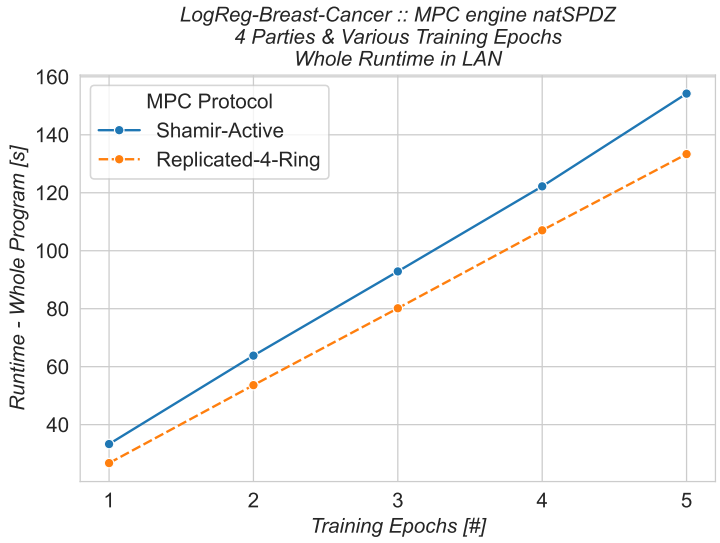

Fig. 6: Overall runtime in seconds for the logistic regression of breast-cancer data in Scenario Two, "Protocol Epochs". We use Replicated and malicious Shamir for 4 parties.

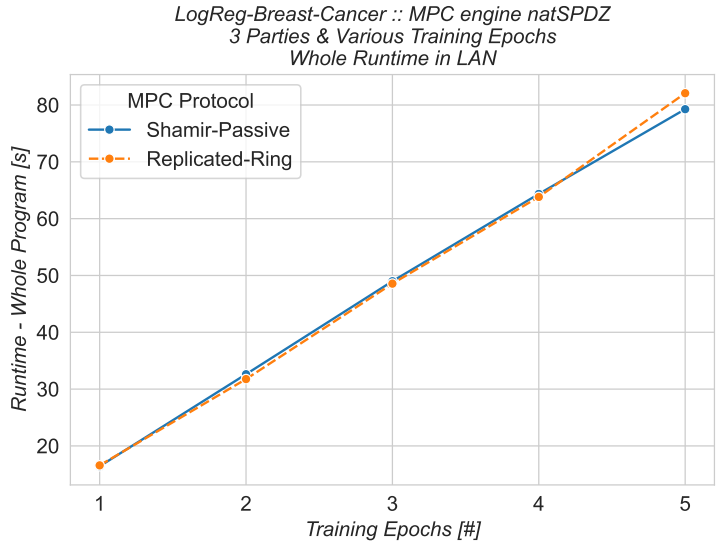

Fig. 7: Overall runtime in seconds for the logistic regression of breast-cancer data in Scenario Two, "Protocol Epochs". We use Replicated and semi-honest Shamir for 3 parties.

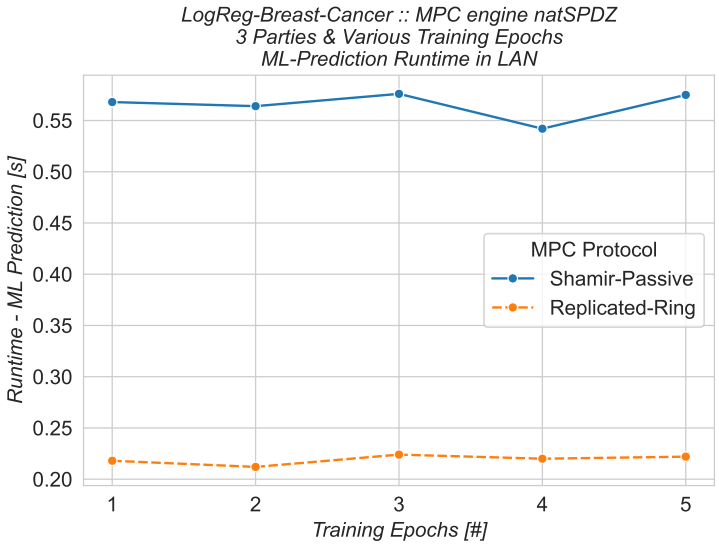

Fig. 8: Evaluation runtime, after training, in seconds for the logistic regression of breast-cancer data in Scenario Two, "Protocol Epochs". We use Replicated and semi-honest Shamir for 3 parties.

5 Conclusion

In this work, we addressed the gap of tedious and complex benchmarking of MPC. Holistic benchmarking of MPC programs within various potential scenarios is usually a cumbersome process; to set up the desired environment and get easy-to-interpret results. Related works so far mostly provide a comparison for certain programs with different engines. For instance, Keller [13] introduces the general-purpose MPC engine `MP-SPDZ`, which allows to select from a variety of MPC protocols (honest/dishonest majority, active/passive corruptions), and compares `MP-SPDZ` with various other MPC engines. However, we need to manually set up the benchmarking environment. Barak et al. [4] provide an MPC benchmark for a variety of settings for one MPC engine but the actual benchmarking is not reproducible.

To the best of our knowledge, for the first time the whole MPC-benchmarking flow is automated, provided by the open-sourced framework `Holistic Benchmarking for MPC` (`b4M`). `b4M` is provides configurations for possibly relevant settings and outputs ready-to-use graphs. Furthermore does it take three relatively-easy steps to add further engines: first, integrate engine-specific commands into `b4M`'s runner class; second, output performance metrics in `b4M`'s format; third, provide a Docker container for the engine's parties.

To showcase `b4M`, we provided two exemplary evaluations for the MPC computation of the dot product of two vectors and logistic regression on breast-cancder data.

Further, we would like to underline once more, the benefit to even have the MPC engines as own benchmark dimension. Especially since the MPC engines can be shipped via Docker containers, the whole setup of any MPC engine within envisioned scenarios should be straightforward and smooth. This way, we envision to make MPC benchmarking as easy as possible for the MPC research community and beyond, which in turn shall lead to an enhanced comparability, usefulness, and reproducibility of MPC benchmarks in projects/papers.

Regarding the consistency, please note that although it is up to the respective researchers/projects to use the same performance evaluations, `b4M` helps making this process as easy as possible. We believe that in the long run such a consistency among the same MPC task/problem area, leads to a smoother development of MPC protocols/evaluations/applications, with an increased quality due to reproducibility.

With this work, we moved towards fully-automated evaluations of MPC engines/protocols/programs. Fully-automated evaluations smoothen the process of setting up and viewing various trade-offs. Hence, such evaluations advance the development of MPC, especially to integrate MPC in further real-life use cases.

Acknowledgments. This work received funding from (i) **PREPARED**, a project funded by the Austrian security research programme KIRAS of the Federal Ministry of Finance (BMF) and (ii) **LICORICE**, a European Union's Horizon Europe project (no. 101168311).

References

1. Abspoel, M., Dalskov, A.P.K., Escudero, D., Nof, A.: An efficient passive-to-active compiler for honest-majority MPC over rings. In: Sako, K., Tippenhauer, N.O. (eds.) Applied Cryptography and Network Security - 19th International Conference, ACNS 2021, Kamakura, Japan, June 21-24, 2021, Proceedings, Part II. Lecture Notes in Computer Science, vol. 12727, pp. 122–152. Springer (2021), doi.org/10.1007/978-3-030-78375-4_6
2. Aly, A., Nawaz, K., Salazar, E., Sucasas, V.: Through the Looking-Glass: Benchmarking Secure Multi-party Computation Comparisons for ReLU 's. In: Beresford, A.R., Patra, A., Bellini, E. (eds.) Cryptology and Network Security. pp. 44–67. Lecture Notes in Computer Science, Springer International Publishing, Cham (2022). https://doi.org/10.1007/978-3-031-20974-1_3
3. Araki, T., Furukawa, J., Lindell, Y., Nof, A., Ohara, K.: High-throughput semi-honest secure three-party computation with an honest majority. In: Weippl, E.R., Katzenbeisser, S., Kruegel, C., Myers, A.C., Halevi, S. (eds.) Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, Vienna, Austria, October 24-28, 2016. pp. 805–817. ACM (2016), doi.org/10.1145/2976749.2978331
4. Barak, A., Hirt, M., Koskas, L., Lindell, Y.: An end-to-end system for large scale p2p mpc-as-a-service and low-bandwidth mpc for weak participants (2018). https://doi.org/https://doi.org/10.1145/3243734.3243801, report Number: 751
5. Buchsteiner, T., Koch, K.W., Rotaru, D., Rechberger, C.: webSPDZ: Versatile MPC on the web. Cryptology ePrint Archive, Paper 2025/487 (2025), https://eprint.iacr.org/2025/487
6. Cramer, R., Damgård, I., Maurer, U.M.: General secure multi-party computation from any linear secret-sharing scheme. In: Preneel, B. (ed.) Advances in Cryptology - EUROCRYPT 2000, International Conference on the Theory and Application of Cryptographic Techniques, Bruges, Belgium, May 14-18, 2000, Proceeding. Lecture Notes in Computer Science, vol. 1807, pp. 316–334. Springer (2000), doi.org/10.1007/3-540-45539-6_22
7. cryptobiu: Mpc simulation framework, https://github.com/cryptobiu/MATRIX
8. Dalskov, A., Escudero, D., Keller, M.: Secure evaluation of quantized neural networks. Proceedings on Privacy Enhancing Technologies (2020), https://petsymposium.org/popets/2020/popets-2020-0077.php
9. Dalskov, A.P.K., Escudero, D., Keller, M.: Fantastic four: Honest-majority four-party secure computation with malicious security. In: Bailey, M.D., Greenstadt, R. (eds.) 30th USENIX Security Symposium, USENIX Security 2021, August 11-13, 2021. pp. 2183–2200. USENIX Association (2021), usenix.org/conference/usenixsecurity21/presentation/dalskov
10. Dobraunig, C., Grassi, L., Guinet, A., Kuijsters, D.: Ciminion: Symmetric encryption based on toffoli-gates over large finite fields. In: Canteaut, A., Standaert, F.X. (eds.) Advances in Cryptology – EUROCRYPT 2021. p. 3–34. Lecture Notes in Computer Science, Springer International Publishing, Cham (2021). https://doi.org/10.1007/978-3-030-77886-6_1
11. Grassi, L., Rechberger, C., Rotaru, D., Scholl, P., Smart, N.P.: Mpc-friendly symmetric key primitives. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security. p. 430–443. CCS '16, Association for Computing Machinery, New York, NY, USA (Oct 2016). https://doi.org/10.1145/2976749.2978332

12. Grassi, L., Øygarden, M., Schofnegger, M., Walch, R.: From farfalle to megafono via ciminion: The prf hydra for mpc applications. In: Hazay, C., Stam, M. (eds.) Advances in Cryptology – EUROCRYPT 2023. p. 255–286. Lecture Notes in Computer Science, Springer Nature Switzerland, Cham (2023). https://doi.org/10.1007/978-3-031-30634-1_9, https://eprint.iacr.org/2022/342
13. Keller, M.: MP-SPDZ: A versatile framework for multi-party computation. In: Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (2020). https://doi.org/10.1145/3372297.3417872
14. Keller, M.: MP-SPDZ: A versatile framework for multi-party computation. In: Ligatti, J., Ou, X., Katz, J., Vigna, G. (eds.) ACM CCS 2020. pp. 1575–1590. ACM Press (Nov 2020). https://doi.org/10.1145/3372297.3417872
15. Keller, M., Orsini, E., Scholl, P.: MASCOT: Faster malicious arithmetic secure computation with oblivious transfer. Cryptology ePrint Archive, Paper 2016/505 (2016). https://doi.org/10.1145/2976749.2978357, https://eprint.iacr.org/2016/505
16. Keller, M., Scholl, P., Smart, N.P.: An architecture for practical actively secure MPC with dishonest majority. In: Sadeghi, A.R., Gligor, V.D., Yung, M. (eds.) ACM CCS 2013. pp. 549–560. ACM Press (Nov 2013). https://doi.org/10.1145/2508859.2516744
17. Lindell, Y.: Secure multiparty computation. Commun. ACM **64**(1), 86–96 (2021), doi.org/10.1145/3387108
18. Lorünser, T., Wohner, F.: Performance Comparison of Two Generic MPC-frameworks with Symmetric Ciphers. p. 587–594 (Apr 2023), https://www.scitepress.org/Link.aspx?doi=10.5220/0009831705870594
19. Marcel Keller: MP-SPDZ: Versatile Framework for Multi-Party Computation @ GitHub, https://github.com/data61/MP-SPDZ
20. Maurer, U.: Secure multi-party computation made simple. Discrete Applied Mathematics **154**(2), 370–381 (Feb 2006). https://doi.org/10.1016/j.dam.2005.03.020
21. Rotaru, D.: awesome-mpc (Nov 2023), https://github.com/rdragos/awesome-mpc
22. Schoenmakers, B.: MPyC—Python package for secure multiparty computation. In: Workshop on the Theory and Practice of MPC (2018), https://github.com/lschoe/mpyc
23. Shamir, A.: How to share a secret. Communications of the ACM **22**(11), 612–613 (Nov 1979). https://doi.org/10.1145/359168.359176
24. Shamir, A.: How to share a secret. Commun. ACM **22**(11), 612–613 (1979). https://doi.org/10.1145/359168.359176, https://doi.org/10.1145/359168.359176
25. Smart, N.P.: Computing on encrypted data. IEEE Secur. Priv. **21**(4), 94–98 (2023), doi.org/10.1109/MSEC.2023.3279517